

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_2ae09dfb-e3e\agent-af067b32601f96bc1.jsonl`  
- SHA-256: `22499f9e6d29bf7c337d7b4f68d8bfb1ffe5a1282cc0892a10237cbb271ad11c`  
- Source modified: `2026-06-13T20:00:01+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_2ae09dfb-e3e`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:55:36.018Z)

Design the GPU-run recipe for P3 on this machine (RTX 2070 Super, 8GB, WSL2 Ubuntu). Read backend/pipeline/detectors/person_detector.py (the torchvision detector cost), wasb_runner.py, and infer from CODE_MAP + the run scripts. Person detection runs over shuttle-seeded candidate windows only (bounded), reusing the EXISTING golden trajectory CSVs (no WASB re-run needed ? trajectories already exist). Report: realistic cost/time for person detection over the 6 golden videos' candidate windows; how to run it headless + keep-awake (Modern Standby kills long runs ? see the keep_aware recipe); cache/output hygiene; how to monitor progress; and whether to run a lightweight offline person-feature extractor (preferred ? no Gemini, no WASB re-run) vs the full FusionSegmenter. Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. Read the ACTUAL files, quote file:line. The fusion P2 segmenter (FusionSegmenter), fusion_features.py, and PersonDetector.detections_per_frame are ALREADY on master. The experiment harness (backend/eval/experiment.py: Variant/compare/compare_unlabeled/load_video_candidates) is shipped. Be precise.

2. assistant (2026-06-13T19:55:38.029Z)

I'll analyze the GPU-run recipe for P3 person detection on your RTX 2070 Super. Let me read the key files to understand the pipeline, cost, and approach.

3. user (2026-06-13T19:55:38.293Z)

```
1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.  
2  
3      Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):  
4      the **person cue is a swappable producer**, not logic baked into one segmenter. Today  
5      `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-  
6      identically);  
7      next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and  
8      later the learned classifier reads its persisted features. Keeping detection here (not  
9      in  
10     the segmenter) is what makes "add a cue = register a producer" true.  
11  
12     Behaviour is unchanged from the in-lined version ? only the home moved:  
13     - `lazy_load_model`           ? load a permissively-licensed torchvision detector  
14     - `make_tracker`             ? a fresh supervision ByteTrack per chunk  
15     - `detect_and_track`         ? one frame ? confident persons with persistent IDs  
16     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows  
17     with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)  
18     that feed the candidate `stats` JSON and the learned fusion features.  
19  
20     The model libs (torchvision, supervision) are imported lazily inside the methods, and  
21     the  
22     detector weights load only on first detection (`lazy_load_model`) ? so importing the  
23     segmenter registry never loads a detector model, and tests can mock the seams. (torch +
```

```

cv2
21 are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22 machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23 (ADR-005; ultralytics' AGPL YOLO was dropped).
24 """
25 import logging
26 from typing import Any, Dict, List, Optional, Tuple
27
28 import cv2
29 import torch
30
31 from backend.utils.geometry import get_velocity_normalised
32
33 logger = logging.getLogger(__name__)
34
35 # torchvision detection models are trained on COCO where the "person" category is
36 # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38 _PERSON_CLASS_ID = 1
39
40 # Permissively-licensed torchvision detectors selectable via config
41 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42 _DETECTOR_FACTORIES = {
43     "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44     "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45     "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46 }
47 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50 class PersonDetector:
51     """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53     Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55     min_action_window_duration}``); the model loads lazily on first use.
56     """
57
58     def __init__(self, config: Dict[str, Any]):
59         self.config = config
60         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61         # `self.model([tensor])` inference call type-checks without a quarantine.
62         self.model: Any = None
63         self.device: Optional[str] = None
64
65     def lazy_load_model(self) -> None:
66         if self.model is not None:
67             return
68
69         # Imported lazily so the module imports without torchvision present and so
70         # the test suite (which pre-sets self.model) never pulls in model weights.
71         from torchvision.models import detection as tv_detection
72
73         idx_cfg = self.config.get("indexing", {})
74         use_gpu = idx_cfg.get("use_gpu", True)
75         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76         if use_gpu and not torch.cuda.is_available():
77             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80         if model_name not in _DETECTOR_FACTORIES:
81             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82         model_name = _DEFAULT_DETECTOR

```

```

83
84         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87         self.model = builder(weights="DEFAULT")
88         self.model.eval()
89         self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99
100     def detect_and_track(self, frame: Any, tracker: Any,
101                          score_thresh: float) -> List[Tuple[int, Tuple[float, float,
102 float, float]]]:
103         """Detect persons in a frame and assign persistent track IDs.
104
105         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
106         This replaces the old ultralytics ``model.track(...)`` call, which bundled
107         detection AND association: torchvision does the detection, supervision's
108         ByteTrack does the association.
109         """
110         import supervision as sv
111
112         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
113         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
114         tensor = torch.from_numpy(rgb).permute(2, 0,
115 1).float().div(255.0).to(self.device)
116         with torch.no_grad():
117             outputs = self.model([tensor])
118
119         out = outputs[0]
120         boxes = out["boxes"].detach().cpu().numpy()
121         labels = out["labels"].detach().cpu().numpy()
122         scores = out["scores"].detach().cpu().numpy()
123
124         # Keep only confident person detections.
125         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
126         if not mask.any():
127             return []
128
129         # ByteTrack requires confidence to be populated (it filters on it internally).
130         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
131 class_id=labels[mask])
132         tracked = tracker.update_with_detections(dets)
133
134         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
135         for i in range(len(tracked)):
136             tid = tracked.tracker_id[i]
137             if tid is None:
138                 continue
139             x1, y1, x2, y2 = tracked.xyxy[i]
140             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
141         return result
142
143     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
144 velocity_thresh: float, merge_gap: float,
145 frame_skip: int = 3,
146 chunk_index: Optional[int] = None) ->
147 List[Dict[str, Any]]:

```

```

144         """
145         Runs person detection + tracking on a video chunk and extracts high-motion
146         action windows. Returns a list of dicts in the full video's timeframe, each
with:
147             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148             track_id_count, chunk_index.
149
150         Args:
151             frame_skip: Process every Nth frame (1 = every frame). Higher values
152                         dramatically reduce inference cost with minimal quality loss.
153             chunk_index: Index of the source chunk (recorded on each window for
traceability).
154         """
155         cap = cv2.VideoCapture(chunk_path)
156         if not cap.isOpened():
157             logger.error(f"Could not open {chunk_path}")
158             return []
159
160         fps = cap.get(cv2.CAP_PROP_FPS)
161         if fps <= 0:
162             fps = 30.0
163
164         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165         if frame_width <= 0:
166             frame_width = 1280.0 # Sensible fallback
167
168         # Effective sampling rate after frame-skipping
169         effective_fps = fps / max(frame_skip, 1)
170
171         # A fresh tracker per chunk ? track IDs only need to be consistent within a
172         # chunk (windows are computed per chunk, then merged across chunks by time).
173         tracker = self.make_tracker(effective_fps)
174         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177         # so windows can be enriched with motion stats for logging + fine-tuning.
178         active_frames = []
179         frame_idx = 0
180         track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182         while True:
183             ret, frame = cap.read()
184             if not ret:
185                 break
186
187             # Skip frames for performance ? at 30fps, every 3rd frame still
188             # gives ~10 samples/second, plenty for human-scale velocity.
189             if frame_idx % max(frame_skip, 1) != 0:
190                 frame_idx += 1
191                 continue
192
193             # Detect persons + assign persistent track IDs.
194             detections = self.detect_and_track(frame, tracker, score_thresh)
195
196             frame_peak_velocity = 0.0
197             frame_active_ids = set()
198             current_ids = set()
199
200             for track_id, bbox in detections:
201                 current_ids.add(track_id)
202
203                 if track_id in track_history:
204                     prev_bbox = track_history[track_id]
205                     velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,

```

```

frame_width)
206             if velocity > velocity_thresh:
207                 frame_peak_velocity = max(frame_peak_velocity, velocity)
208                 frame_active_ids.add(track_id)
209
210             track_history[track_id] = bbox
211
212             # Prune stale IDs that are no longer visible ? prevents false
213             # velocity spikes when the tracker reassigns an old ID.
214             track_history = {k: v for k, v in track_history.items() if k in current_ids}
215
216             if frame_active_ids:
217                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
218
219             frame_idx += 1
220
221         cap.release()
222
223         # Group active frames into continuous windows
224         if not active_frames:
225             return []
226
227         max_gap_frames = int(merge_gap * fps)
228
229         def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
230             velocities = [v for _, v, _ in group]
231             track_ids = set().union(*[ids for _, _, ids in group])
232             return {
233                 "start": group[0][0] / fps + chunk_start,
234                 "end": group[-1][0] / fps + chunk_start,
235                 "active_frame_count": len(group),
236                 "peak_velocity": max(velocities),
237                 "mean_velocity": sum(velocities) / len(velocities),
238                 "track_id_count": len(track_ids),
239                 "chunk_index": chunk_index,
240             }
241
242         windows = []
243         group = [active_frames[0]]
244         for af in active_frames[1:]:
245             if af[0] - group[-1][0] <= max_gap_frames:
246                 group.append(af)
247             else:
248                 windows.append(_finalize(group))
249                 group = [af]
250         windows.append(_finalize(group))
251
252         # Filter out extremely short action windows
253         min_window_duration = self.config.get("indexing",
254         {}).get("min_action_window_duration", 2.0)
255         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257         return windows
258
259     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
260                             frame_skip: int = 3) -> Dict[str, Any]:
261         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
262         video.
263         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
264         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
265         discards the
266         per-frame boxes), this returns the raw per-frame detections so the fusion
267         features can
268         align them with the shuttle trajectory. It reads the source by **absolute frame

```

```

index**
266         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
267         the WASB trajectory's ``.frame`` exactly ? that alignment is what
268         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
269
270         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
271         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
seek
272         accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
273         treat the indices as  $\pm$ a few frames on real footage ? fine for window-level
features.
274         ""
275         self.lazy_load_model()
276         cap = cv2.VideoCapture(video_path)
277         if not cap.isOpened():
278             logger.error(f"Could not open {video_path}")
279             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
280
281         fps = cap.get(cv2.CAP_PROP_FPS)
282         if fps <= 0:
283             fps = 30.0
284         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
285         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
286         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
287         step = max(frame_skip, 1)
288         tracker = self.make_tracker(fps / step)
289
290         start_frame = max(0, int(start_frame))
291         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
292         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
293         frame_idx = start_frame
294         while frame_idx <= end_frame:
295             ret, frame = cap.read()
296             if not ret:
297                 break
298             if (frame_idx - start_frame) % step == 0:
299                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
300                 frame_idx += 1
301         cap.release()
302         return {"frames": frames, "fps": fps,
303                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
304

```

4. user (2026-06-13T19:55:38.475Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

5. user (2026-06-13T19:55:38.797Z)

docs\CODE_MAP.md

6. user (2026-06-13T19:55:38.831Z)

```

run_all_tests.py
run_eval.py
run_phase3_eval.py
run_regression.py
backend\reporting\run_signals.py
scratch\run_gx010125.py
run_process_and_stitch.py

```

7. assistant (2026-06-13T19:55:40.360Z)

Let me continue reading the relevant files.

8. user (2026-06-13T19:55:41.233Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-151 of 348 total (33984 tokens, cap 25000). Call Read with offset=152 limit=151 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
4      hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
5      segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
6
7      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
8      doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
9      (this revision follows #40).
10
11      ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust
12      git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no
13      FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via
14      `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not
15      basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`
16      defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
```

`match\_intervals` requires real overlap (iou>0); `run\_regression` baseline lives in tracked
`eval\_baselines`/` and **exits 3 on a missing baseline**;

the API validates
`video\_path`/`output\_filename`. New tests: `test\_api\_endpoints`, `test\_compiler`,
`test\_wasb\_runner`, `test\_reingest\_safety`, `test\_detector\_keying`, `test\_config\_wasb`,
`test\_path\_safety`, `test\_reliability\_hardening` (+ collector `test\_licensing\_gate`,
`test\_import\_guard`, `test\_cvat\_robustness`).

```
8
9      **LEGEND**
10      - `@path` = file (always repo-relative)
11      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12      - `->` = dataflow / call / "produces"
13      - `$` = data contract (canonical shape)
14      - `?` = gotcha / footgun
15      - `?` = registry / plugin extension point
16      - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18      ---
19
20      ## 1. PIPELINE
21
22      ### 1A. Runtime: video file -> highlight reel
23      ```
24      typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
25      defaults, never fatal)
26
27      @backend/config/loader.py::load_config ->
28      @backend/config/models.py::AppConfig
29      -> video_id = MD5(whole file) @backend/utils/hashing.py::compute_video_id
30      (single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
31      copies)
32      -> validate
33      @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
34      FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
35      -> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
36      @backend/pipeline/validators/__init__.py)
37      [validation FAIL -> persist status="failed", return early; report STILL emitted
38      in finally]
39      -> db.add_video(video_id, filepath, status, [r.dict()])
40      @backend/infrastructure/database.py
```

```

31     -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
32     -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
33     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
34     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
35     -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
36     -> select segment ids -> [(start,end)] (+ stitching padding)
37     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
38     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
39     ...
40     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
41     ...
42     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
43     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
44     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
45     -> P4 db.add_play_segment + db.link_candidate_to_segment
46     ...
47     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
48     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
49
50     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
51     ...
52     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
53     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
54     -> WasbRunner.run_predict(video, out_dir)
55     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
56     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
57     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
58     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
59     ...
60     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
61
62     ### 1C. Calibration / eval flow (NO pipeline run unless told)
63     ...
64     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
65     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
66     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
67     -> harness.evaluate -> EvalReport -> report_to_dict
68     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
69     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
70     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
71     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py

```

```

72     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
73     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
74     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
75     ...
76     Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
77
78     ---
79
80     ## 2. SUBSYSTEMS
81
82     ### 2.0 Orchestration & config
83     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
84     - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
85     - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests
monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
86     - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST
/api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate ->
`{upload_id}`, `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY
sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach
-> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble
-> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs
/api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state
(`::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits);
per-user partitioning lands with PR-3. `::download_highlight` `@GET
/api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then
FileResponse stream (the old /api/compile alert path was browser-unreachable). Config:
`SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95,
allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
87     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`) ? kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
88     - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
89     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
90     - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`

```

```

(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
91     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
92     - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
93     - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
94
95     - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
96     - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
97     - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
98     - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
99     - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`).
100    - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`,
`min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config` = `AppConfig` alias.
101    - `@backend/config/loader.py` ? load/save with defaults-merge.
102    - `::DEFAULT_CONFIG_PATH` = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_built_in_defaults` = `AppConfig().model_dump(...)`).
103    - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
104    - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
105    - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
106    - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python>3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).

```

107

108 `### 2.1 Segmenters (under pipeline/segmenters) ?`

109 `- `@backend/pipeline/segmenters/base.py``

110 `- `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract`
``name`,`description`,`process_video`. $ `process_video(video_id, video_path, player_pool=None,`
`max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.`
111 `- `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new`
`segmenter**`: subclass `BasePlaySegmenter`, call `segmenter\_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/\_\_init\_\_.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` ->
name/description/`\_\_init\_\_` MUST NOT touch `self.db`/`self.config`. ? `list\_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).

112 `- `@backend/pipeline/segmenters/__init__.py` ? imports all 6`

(`motion,gemini,yolo\_hybrid,tracknet\_hybrid,wasb\_hybrid,fusion\_hybrid`) = the de-facto
registration manifest; re-exports them + `segmenter\_registry` via `\_\_all\_\_`.

113 `- `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ?`
SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix
BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider
handoff -> P4 DB/link; single `::shot\_count\_from` (old wasb/tracknet shot_count drift gone).
NOT registered itself. ? Subclass contract: `family` (config section

`indexing.<family>.velocity\_threshold(0.02)`, `output/<family>` trajectory dir,
`<family>-hybrid-<model>` metadata tag), `display\_label` (log lines), `name`/`description`
properties, `\_build\_runner(idx\_cfg) -> (DetectorRunner, detection\_params extras)`. ****Two fusion**
seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test):

`::enrich\_candidate\_stats(cw,points,fps,frame\_width,video\_path,idx\_cfg)` **** (base = the 4**
motion-key stats dict; computed once into `window\_stats`, used for persist + the handoff gate)
****and `::select\_for\_handoff(windows>window\_stats,idx\_cfg)` **** (base = all indices). ? shared**
knobs still YOLO-named (`log\_yolo\_candidates/store\_yolo\_candidates`); whole-video single pass;
needs `{PROVIDER}\_API\_KEY` (missing -> []). Equivalence pinned by
`tests/test\_trajectory\_hybrid\_equivalence.py` (detector identity, shot_count, offsets, + the
base-hook-identity tests ? parametrized over both twins).**

114 `- `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name`
``'fusion_hybrid'`, **default-OFF** multi-signal fusion P2) ? subclasses the engine;`
``family='fusion'`, `display_label='Fusion'`. `::_build_runner` selects the substrate via`
``indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config.`
``::enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via`
``rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per`
trajectory) and, only when `fusion.cue\_flags.person`, the ~5 person features
(`::\_person\_features` ? lazy `PersonDetector.detections\_per\_frame` over the window's ORIGINAL-
video frame range ? `fusion.features.compute\_person\_features`). `::select\_for\_handoff`
overrides to apply served Gate A (`fusion.min\_crossings`) ? Gate B (`fusion.min\_players`,
person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full
superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a
tuned wasb run, set `fusion.velocity\_threshold = wasb.velocity\_threshold` (the engine reads
`idx\_cfg['fusion'].velocity\_threshold`). ? person features eager even for gated windows (P3
cleanup). Tests: `tests/test\_fusion\_hybrid.py`.

115 `- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name`
``'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin`
subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection\_params` extras =
`weights\_path` only.

116 `- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name`
``'tracknet_hybrid'`) ? thin subclass: `family='tracknet'`, wires`
``TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.`

117 `- `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name`
``'yolo_hybrid'`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to`
``detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now`
ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps
sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to
`self.person` (`PersonDetector`). Imports `\_DEFAULT\_DETECTOR` from person_detector for the
`detection\_params` snapshot. ? `yolo\_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini`*
config keys are deprecated aliases still honored; still mypy-quarantined for pre-existing
orchestration debt (the torchvision code that caused it left).

118 `- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?`
pure-AI, no P2. Chunks are RE-ENCODED + downscaled to `indexing.ai\_chunk\_scale\_width` (default

```

1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's
`MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes
keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are
aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report
rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
119 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
120
121     ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
122 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
123 - `@backend/pipeline/detectors/tracknet_runner.py`
124 - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
125 - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled),
`::TrajectoryPoint` (frame,visible,x,y).
126 - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
127 - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `wsl_bash` now shared via `detectors/base.py::WslCommandMixin` (2026-06-11 dedupe).
Both runners now satisfy the DetectorRunner ABC for future swappability. **Cache hygiene
(#109):** on a SUCCESSFUL run both runners delete their WSL intermediates ? WASB the frame cache
+ staged video, TrackNet the staged video ? gated by `indexing.{wasb,tracknet}.keep_frames`
(default false; on FAILURE kept so a re-run resumes). Pre-run `min_free_gb` disk-guard warning.
Shared helpers `WslCommandMixin::wsl_rm_rf`/`_wsl_free_gb`.
128 - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.jsonl'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text`/`atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
129 - `@backend/pipeline/detectors/person_detector.py::PersonDetector` ? the **PERSON rally
cue** (torchvision detector + supervision ByteTrack + velocity windowing), extracted from
yolo_hybrid (fusion P1; type-clean, NOT quarantined). `::PERSON_CLASS_ID=1` ? (ultralytics used
0; wrong value -> zero detections),
`::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. `::lazy_load_model`
(lazy `torchvision` import so module/tests load without it; `self.model` typed `Any`),
`::make_tracker` (per-chunk ByteTrack ? IDs chunk-local), `::detect_and_track` (one frame ->
confident persons w/ track IDs; BGR->RGB, COCO person filter, ByteTrack association),
`::extract_action_windows_from_chunk` (chunk -> high-motion windows w/ `peak/mean_velocity`,
`active_frame_count`, `track_id_count` ? the stats that feed candidate `stats` AND the future
learned-fusion person features). ? velocity at `effective_fps = fps/frame_skip` ->
`yolo_velocity_threshold` coupled to `frame_skip`. Consumed by yolo_hybrid AND the fusion
segmenter (P2 reuses the SAME producer over shuttle-seeded windows via `::detections_per_frame`

```

```

? add-a-producer, not a refactor). BSD torchvision + MIT ByteTrack.
**::detections_per_frame(video_path,start_frame,end_frame,frame_skip)** (fusion P2) reads the
ORIGINAL video by ABSOLUTE frame range (seek+sequential) ? {frame_idx: [(track_id,bbox)]} +
`fps/frame_width/frame_height`, so indices align 1:1 with the WASB trajectory `.frame` (no clip
re-timing). ? cv2 frame-seek accuracy varies by codec (±a few frames on real footage).
130 - `@backend/pipeline/detectors/fusion_features.py` ? **pure** per-window person-cue
feature math (no I/O/torch/cv2; mirrors rally_gate discipline). Inputs: per-frame
`{frame_idx:[(tid,bbox)]}` + the in-window shuttle points + optional normalized `court_polygon`
+ `net=(axis,pos)`. `::point_in_polygon` (ray-cast), `::person_in_court` (foot-point; None
polygon=count-all; bad dims=False), `::in_court_counts`, `::players_in_court` (median),
`::in_court_ratio`, `::mean_player_motion` (per-track centre displacement; x by width, y by
height), `::two_sided_motion` (both net-halves occupied), `::shuttle_player_colocation` (shuttle
inside a person box ±`pad_frac` halo over aligned visible frames ? **the held-shuttle
discriminator**), `::compute_person_features` (aggregator ? the 5 `::PERSON_FEATURE_NAMES`). All
scale/translation-invariant, graceful to None/empty. Tests: `tests/test_fusion_features.py`.
131 - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds;
**estimate=`param` lets a caller scoring many windows pass a precomputed `(axis,net,span)` to
skip per-window re-estimation ? used by FusionSegmenter).
`::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is
`(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments / fusion_hybrid.
132
133     ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
134 - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
135 - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
136 - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs 22 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>~0.1` = overfit alarm.
137 - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfigs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` (`$start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? {load_golden_gts,
make_predict_fn, BASELINE} imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
138 - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfigs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
139 - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
140 - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
141 - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`); ? unknown
stage raises ValueError; reused by regression.py),

```

```

`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
142 - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
143 - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON "model":"logreg" tag.
144 - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPS ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
145 - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO.
`::build_frame_arrays` (takes `fps`; speed is per-SECOND + occlusion-gap-guarded since the
2026-06-10 sweep), `::features` (scipy `uniform/maximum_filter1d` windowed), `::labels_for`,
`::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based with **midrank
ties**, scipy.stats.rankdata), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold
F1). ? DIRECTIONAL only (n=6, linear, threshold-transfer unsolved); held-out per-frame AUC
0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). **n=6 LOVO:
learned 0.626 vs heuristic 0.480** (learned rose from 0.583 after the fps-normalization fix).
Manifest `output/human_lovo_manifest.json` (n=6; trajectory paths now under `Annotation
Setup/Collect/Trajectories/`) shared w/ calibrate_local.
146 - ? **MULTI-COURT / background-game confound (2026-06-08 finding)**: **WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-
time visibility "contrast" (?3.6x heuristic works, ?1.9x fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
147 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
148 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
149 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json` (tracked, NOT under gitignored
output/), `::regress`, `::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->`FileProvider`/"vendor"->`HumanVendor`/"auto"->`oracle`),
`::save_baseline`/`::load_baselines`, `::gt_hash` (label-set fingerprint, reused by
experiment.py), `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass). CLI
`run_regression.py` exit 0 ok / 1 regress / 2 no-DB / 3 no-baseline = the promotion gate.
150 - `@backend/eval/experiment.py` ? A/B + shadow + **SxS** variant layer (the thin
"experiment" glue per `docs/EXPERIMENT_HARNESS.md`; composes
metrics/calibration/classifier/training/regression ? NO new scoring math).
`::Variant` (segmenter,config_overrides,cue_flags,`min_crossings`[Gate A],`min_players`[Gate
B],classifier_model,feature_names,threshold,merge_gap;
`to_dict/from_dict/spec_hash/is_learned`), `::CONTROL` (pinned zero-gate baseline).

```

```
`::VideoCandidates`(name,intervals,features{col-major},gts), `::predict` (gate?learned-
filter?'gemini_refine.merge_overlaps'), `::evaluate_variant` (static / pinned-model / **honest-
LOVO** train-on-others ? mirrors distill_local), `::compare` (labeled **B?A** deltas +
`label_set_hash`), `::compare_unlabeled` (**NEW-footage SxS**: agreed/a_only/b_only via
`match_intervals`, no GT ? the owner's "SxS on new videos"), `::record_experiment` (?
`eval_baselines/experiment_leaderboard.json`), `::load_video_candidates` (DB bridge via
`query_candidate_segments`). ? enabling a gate needs its feature persisted (absent?0.0?fails
gate, warns); learned variant on an unlabeled video needs a pinned `classifier_model`. **No-reg
invariant (tested): all cues OFF ? byte-identical to CONTROL.**
151
```

9. user (2026-06-13T19:55:41.742Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/fusion_features.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py
```

10. user (2026-06-13T19:55:41.759Z)

```
backend\pipeline\detectors\fusion_features.py
```

11. user (2026-06-13T19:55:41.774Z)

```
backend\eval\experiment.py
```

12. user (2026-06-13T19:55:41.872Z)

No files found

13. assistant (2026-06-13T19:55:43.294Z)

Let me read the fusion_features and experiment files to complete the picture.

14. user (2026-06-13T19:55:43.518Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of **scale/translation-invariant**
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in **pixels** ; `(track_id, box)` when motion
needs it.
12     - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14     - A shuttle point has `frame`, `visible`, `x`, `y` (pixels) ? the WASB
trajectory shape.
15     - `court_polygon` is normalised 0-1 `[(x, y), ...]` or `None` (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21     """
```

```

22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79
80     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
81                        frame_height: float,
82                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:

```

```

82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85 court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110 frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112 normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113 not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131 frame_height: float, net: Optional[Net],
132 court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137         0.0.
138         """
139         if net is None or not per_frame:
140             return 0.0
141         axis, pos = net
142         both = 0
143         for _frame_idx, dets in per_frame.items():
144             near, far = False, False

```



```

204             min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216

```

15. user (2026-06-13T19:55:43.983Z)

```

1         """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3         This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4         identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5         no-regression gate, the pinnable classifier artifact, the persisted-candidate
6         substrate) already exists ? this module composes it. No new scoring math lives
7         here; everything defers to:
8
9         - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`   ? `pct_improvement`
15
16        The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17        (per-frame signals ? run once on the GPU/WSL box, persisted into
18        `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19        signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20        given persisted candidate features it produces `List[Interval]` deterministically,
21        with no GPU and zero production risk. So most experiments are pure offline
22        re-scoring of stored data and never touch the served pipeline.
23
24        Three modes (`EXPERIMENT_HARNESS.md` §5):
25        - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26          aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27          held-out loop, but variant-parameterised.
28        - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29          variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30          human spot-checks (and what two highlight reels would show side by side).
31        - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32          (exit 0/1/3); rollback = flip the variant/config, never `git revert`.**
33
34        Pure + offline + unit-tested. The only DB-touching helper is
35        `load_video_candidates`, which reads persisted candidates via
36        `Database.query_candidate_segments`.
37        """
38        from __future__ import annotations
39
40        import json
41        import logging
42        import os
43        from dataclasses import dataclass, field
44        from datetime import datetime, timezone
45        from hashlib import sha1
46        from typing import Any, Dict, List, Optional, Sequence
47

```

```

48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54
55 logger = logging.getLogger(__name__)
56
57 # Where a comparison run appends one reproducible record. Tracked location (NOT under
58 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59 # regression.DEFAULT_BASELINE_PATH.
60 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
61 _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
64 class ExperimentError(ValueError):
65     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66     score an unlabeled video with no pinned model, or a gate referencing an absent
67     feature)."""
68
69
70 # ----- Variant
71
72
73 @dataclass
74 class Variant:
75     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77     A variant turns one video's persisted candidate windows + features into rally
78     intervals. Every knob defaults to the control behaviour (no gate, no learned
79     filter), so a variant that merely carries a new, disabled cue is byte-identical
80     to control ? the core no-regression guarantee.
81
82     Fields:
83     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84       for the leaderboard and for selecting the served path (the existing
85       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
86       New cues are recorded here default-OFF.
87     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
88       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
90 shuttle windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
92       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93       cue persists that feature).
94     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
95       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
96       required for ``compare_unlabeled`` on a learned variant.
97     - ``feature_names`` ? the persisted features the learned filter consumes. When set
98       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
99       (honest) ? the recipe, not a pinned artifact.
100     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
102     """
103
104     name: str
105     segmenter: str = "wasb_hybrid"
106     config_overrides: Dict[str, Any] = field(default_factory=dict)
107     cue_flags: Dict[str, bool] = field(default_factory=dict)
108     min_crossings: int = 0
109     min_players: int = 0
110     classifier_model: Optional[str] = None
111     feature_names: Optional[List[str]] = None

```

```

112     threshold: float = 0.5
113     merge_gap: float = 1.0
114
115     def is_learned(self) -> bool:
116         """True if this variant applies a learned classifier (pinned model or
recipe)."""
117         return self.classifier_model is not None or bool(self.feature_names)
118
119     def to_dict(self) -> dict:
120         return {
121             "name": self.name,
122             "segmenter": self.segmenter,
123             "config_overrides": self.config_overrides,
124             "cue_flags": self.cue_flags,
125             "min_crossings": self.min_crossings,
126             "min_players": self.min_players,
127             "classifier_model": self.classifier_model,
128             "feature_names": self.feature_names,
129             "threshold": self.threshold,
130             "merge_gap": self.merge_gap,
131         }
132
133     @classmethod
134     def from_dict(cls, d: dict) -> "Variant":
135         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
136         return cls(**{k: v for k, v in d.items() if k in known})
137
138     def spec_hash(self) -> str:
139         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
and makes a result reproducible. The pinned **control** variant always hashes
the
141         same, so a regression is always traceable to a recipe change, never to drift."""
142         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
143         return sha1(blob.encode()).hexdigest()[:12]
144
145
146     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147     #: filter. Always the comparison reference; "rollback" = make this the served variant.
148     CONTROL = Variant(name="control")
149
150

```

16. user (2026-06-13T19:55:44.407Z)

```

1     """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3     Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4     processes each video through a segmenter (so detections land in the indexer DB),
5     scores each against its ground-truth rally CSV, and prints per-video plus
6     corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8     Examples
9     -----
10    Free CPU baseline on a single video (validate the loop):
11        python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12            --segmenter motion --only shuttlest_1
13
14    Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15        python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
16            --json output/phase3_report.json
17
18    Re-score only (videos already processed), no re-segmentation:
19        python run_phase3_eval.py --manifest ../manifest.json --score-only
20    """

```

```

21
22     import os
23     import sys
24     import json
25     import time
26     import argparse
27     import logging
28
29     from dotenv import load_dotenv
30     load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32     from backend.infrastructure.database import Database
33     from backend.config import load_config
34     from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
35     # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36     from backend.pipeline.segmenters import segmenter_registry
37     from backend.eval.annotations import load_annotations
38     from backend.eval.harness import evaluate, format_report_text, report_to_dict
39     from backend.eval import batch
40
41     logger = logging.getLogger(__name__)
42
43
44     def build_parser() -> argparse.ArgumentParser:
45         p = argparse.ArgumentParser(
46             description="Batch process + evaluate a collector export manifest (Phase 3).",
47             formatter_class=argparse.RawDescriptionHelpFormatter,
48         )
49         p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
50         p.add_argument("--videos-root", default=None,
51             help="Root for resolving manifest relative local_paths "
52             "(default: the collector repo root, two levels above the
manifest).")
53         p.add_argument("--segmenter", default="motion",
54             help="Segmenter to run (default: motion ? free/CPU). "
55             "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
56         p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
57             help="Which prediction stage(s) to score (default: auto by
segmenter).")
58         p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
59         p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
60         p.add_argument("--only", action="append", default=None,
61             help="Only this video_id (repeatable).")
62         p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
63         p.add_argument("--max-frames", type=int, default=None,
64             help="Cap segmenter frames (fast validation).")
65         p.add_argument("--score-only", action="store_true",
66             help="Skip segmentation; only score detections already in the DB.")
67         p.add_argument("--reprocess", action="store_true",
68             help="Re-run segmentation even if the video already has segments.")
69         p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
70         p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
71         p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
72         return p
73
74
75     def main(argv=None) -> int:
76         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

```

```

77     args = build_parser().parse_args(argv)
78
79     manifest_path = os.path.abspath(args.manifest)
80     if not os.path.exists(manifest_path):
81         print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
82         return 2
83     manifest_dir = os.path.dirname(manifest_path)
84     videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
85
86     with open(manifest_path) as f:
87         manifest = json.load(f)
88     entries = manifest.get("eval_sets", [])
89
90     cfg = load_config() # typed AppConfig (same object the live pipeline feeds
segmenters)
91     db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92     os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93     db = Database(db_path)
94
95     stages = batch.default_stages_for(args.segmenter, args.stage)
96
97     # Resolve the segmenter class up front (unless purely scoring).
98     segmenter = None
99     if not args.score_only:
100         seg_cls = segmenter_registry.get(args.segmenter)

```

17. user (2026-06-13T19:55:44.439Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/config.json

18. assistant (2026-06-13T19:55:46.090Z)

Now let me read the config.json and get the key scripts for running P3.

19. user (2026-06-13T19:55:46.528Z)

```

1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
26             "default_segmenter": "gemini",
27             "use_gpu": true,
28             "motion_threshold": 0.08,

```

```

29         "min_segment_duration": 3.0,
30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_prefetch_workers": 2,
39         "min_action_window_duration": 2.0,
40         "log_yolo_candidates": true,
41         "store_yolo_candidates": true,
42         "ai_handoff_padding": 2.0,
43         "ai_max_workers": 1,
44         "tracknet": {
45             "wsl_distro": "Ubuntu",
46             "repo_dir": "~/models/TrackNetV4",
47             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48             "conda_env": "TrackNetV4",
49             "weights_path": "",
50             "queue_length": 5,
51             "stage_in_wsl": true,
52             "wsl_stage_dir": "~/clips",
53             "timeout_sec": 1800,
54             "velocity_threshold": 0.02
55         },
56         "wasb": {
57             "wsl_distro": "Ubuntu",
58             "repo_dir": "~/models/WASB-SBDT",
59             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60             "conda_env": "wasb",
61             "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62             "sport": "badminton",
63             "wsl_stage_dir": "~/clips",
64             "timeout_sec": 1800,
65             "velocity_threshold": 0.02
66         }
67     },
68     "output": {
69         "default_highlights_name": "highlights.mp4",
70         "db_name": "sports_indexer.db"
71     },
72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

20. user (2026-06-13T19:55:47.292Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/output/wasb_tuned_segments_vs_golden.csv

21. user (2026-06-13T19:55:47.687Z)

```

C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_avi_full_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_avi_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_baseline_gated.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_baseline_segments.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_gemini_fullvideo.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_gemini_refined.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_gemini_smoke.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_min5s_segments.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_tuned_segments.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/gbaaddy_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/kushagra_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/largetestvideo_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/mahadevpura_gemini_refined.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/mahadevpura_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/testlarge_gemini_fullvideo.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/testlarge_gemini_rallies.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/testlarge_vs_human.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/wasb/GX010125_1080p__89605e80ed01_wasb.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/wasb/mahadevpura_smoke_180s__05e0e577029d_wasb.csv
C:/Users/avidu/Projects/badminton-highlight-indexer/output/wasb_tuned_segments.csv

```

22. user (2026-06-13T19:55:48.267Z)

```

backend\pipeline\detectors\person_detector.py:41:# `indexing.detector_model`. All are BSD-
licensed (code + COCO weights).
backend\pipeline\detectors\person_detector.py:54:     ``config`` (reads ``indexing.{use_gpu,
detector_model, detector_score_threshold,
backend\pipeline\detectors\person_detector.py:79:         model_name =
idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
backend\pipeline\detectors\person_detector.py:81:         logger.warning(f"Unknown
detector_model '{model_name}'. Falling back to {_DEFAULT_DETECTOR}.")
backend\pipeline\detectors\person_detector.py:142:
frame_skip: int = 3,
backend\pipeline\detectors\person_detector.py:151:         frame_skip: Process every Nth
frame (1 = every frame). Higher values
backend\pipeline\detectors\person_detector.py:169:         effective_fps = fps / max(frame_skip,
1)
backend\pipeline\detectors\person_detector.py:189:         if frame_idx % max(frame_skip, 1)
!= 0:
backend\pipeline\detectors\person_detector.py:258:     def detections_per_frame(self, video_path:
str, start_frame: int, end_frame: int,
backend\pipeline\detectors\person_detector.py:259:         frame_skip: int =
3) -> Dict[str, Any]:
backend\pipeline\detectors\person_detector.py:287:         step = max(frame_skip, 1)

```

23. assistant (2026-06-13T19:55:49.862Z)

Now let me check for the 6 golden videos, the manifest, and check for any keep-awake patterns in the codebase.

24. user (2026-06-13T19:55:53.497Z)

```

C:/Users/avidu/Projects/badminton-highlight-indexer/artifacts/rally_tal_gen0/0.1.0/manifest.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/wasb/GX010125_1080p__89605e80ed01_wasb_wasbcache/manifest.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/wasb/mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache/manifest.json
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/manifest.json

```

25. user (2026-06-13T19:55:54.994Z)

```

1      {

```

```

2      "commercial_only": true,
3      "statuses": [
4          "curated"
5      ],
6      "total_videos": 7,
7      "total_segments": 264,
8      "eval_sets": [
9          {
10             "video_id": "my_custom_local_match",
11             "sport": "badminton",
12             "csv": "badminton/my_custom_local_match.csv",
13             "local_path": "data/videos/my_custom_match.mp4",
14             "video_url": null,
15             "match_name": "My Custom Local Match",
16             "fps": 30.0,
17             "resolution": "1080p",
18             "license_type": "Proprietary",
19             "is_commercial": true,
20             "status": "curated",
21             "num_segments": 2
22         },
23         {
24             "video_id": "adarsh_avi_singles",
25             "sport": "badminton",
26             "csv": "badminton/adarsh_avi_singles.csv",
27             "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\Adarsh and Avi.MP4",
28             "video_url": null,
29             "match_name": "Adarsh and Avi (singles)",
30             "fps": 59.94,
31             "resolution": "1080p",
32             "license_type": "Proprietary",
33             "is_commercial": true,
34             "status": "curated",
35             "num_segments": 96
36         },
37         {
38             "video_id": "kushagra_singles",
39             "sport": "badminton",
40             "csv": "badminton/kushagra_singles.csv",
41             "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\KushagraYashAviFirstVideo.MP4",
42             "video_url": null,
43             "match_name": "Kushagra Yash Avi (singles, multi-court)",
44             "fps": 59.94,
45             "resolution": "4K",
46             "license_type": "Proprietary",
47             "is_commercial": true,
48             "status": "curated",
49             "num_segments": 32
50         },
51         {
52             "video_id": "largetest_doubles",
53             "sport": "badminton",
54             "csv": "badminton/largetest_doubles.csv",
55             "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\LargeTestVideo.MP4",
56             "video_url": null,
57             "match_name": "LargeTestVideo (doubles)",
58             "fps": 29.97,
59             "resolution": "5.3K",
60             "license_type": "Proprietary",
61             "is_commercial": true,
62             "status": "curated",
63             "num_segments": 49

```

```

64     },
65     {
66         "video_id": "gbaaddy",
67         "sport": "badminton",
68         "csv": "badminton/gbaaddy.csv",
69         "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\gBaaddy20May2026.MP4",
70         "video_url": null,
71         "match_name": "gBaaddy 20 May 2026 (multi-court)",
72         "fps": 59.94,
73         "resolution": "1080p",
74         "license_type": "Proprietary",
75         "is_commercial": true,
76         "status": "curated",
77         "num_segments": 48
78     },
79     {
80         "video_id": "testlarge_short",
81         "sport": "badminton",
82         "csv": "badminton/testlarge_short.csv",
83         "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\TestLargeVideo.MP4",
84         "video_url": null,
85         "match_name": "TestLargeVideo (short)",
86         "fps": 29.97,
87         "resolution": "5.3K",
88         "license_type": "Proprietary",
89         "is_commercial": true,
90         "status": "curated",
91         "num_segments": 6
92     },
93     {
94         "video_id": "mahadevpura_singles",
95         "sport": "badminton",
96         "csv": "badminton/mahadevpura_singles.csv",
97         "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\MahadevpuraSingles.MP4",
98         "video_url": null,
99         "match_name": "Mahadevpura Singles (borderline multi-court)",
100        "fps": 29.97,

```

26. user (2026-06-13T19:55:55.835Z)

```

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/providers/gemini.py:
time.sleep(2 * (attempt + 1))
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/providers/gemini.py:
time.sleep(10)
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/providers/gemini.py:
time.sleep(3 * (2 ** attempt) + random.uniform(0, 2))
C:/Users/avidu/Projects/badminton-highlight-indexer/scratch/gemini_video_debugger.py:
time.sleep(10)
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_async_jobs.py:
time.sleep(0.05)

```

27. user (2026-06-13T19:55:55.997Z)

```

total 12K
drwxr-xr-x 1 avidu 197609    0 Jun 10 15:49 badminton
-rw-r--r-- 1 avidu 197609 3.6K Jun 10 15:49 manifest.json

```

28. assistant (2026-06-13T19:55:57.737Z)

Let me check for the actual golden videos and get their sizes.

29. user (2026-06-13T19:55:58.518Z)

total 38G

```
-rw-r--r-- 1 avidu 197609 11G Jun 1 08:38 Adarsh and Avi.MP4
-rw-r--r-- 1 avidu 197609 4.1K Jun 8 17:20 Adarsh and Avi.rallies.csv
-rw-r--r-- 1 avidu 197609 7.6G May 22 08:02 KushagraYashAviFirstVideo.MP4
-rw-r--r-- 1 avidu 197609 1.4K Jun 8 17:41 KushagraYashAviFirstVideo.rallies.csv
-rw-r--r-- 1 avidu 197609 3.2K Jun 10 17:38 KushagraYashAviFirstVideo.report.json
-rw-r--r-- 1 avidu 197609 1.6K Jun 10 17:38 KushagraYashAviFirstVideo.report.md
-rw-r--r-- 1 avidu 197609 493 Jun 10 17:38 KushagraYashAviFirstVideo.summary.md
-rw-r--r-- 1 avidu 197609 11G May 4 17:18 LargeTestVideo.MP4
-rw-r--r-- 1 avidu 197609 2.1K Jun 8 20:28 LargeTestVideo.rallies.csv
-rw-r--r-- 1 avidu 197609 188M Jan 16 08:00 MahadevpuraSingles.MP4
-rw-r--r-- 1 avidu 197609 1.3K Jun 9 08:08 MahadevpuraSingles.rallies.csv
-rw-r--r-- 1 avidu 197609 3.2K Jun 10 16:29 MahadevpuraSingles.report.json
-rw-r--r-- 1 avidu 197609 1.6K Jun 10 16:29 MahadevpuraSingles.report.md
-rw-r--r-- 1 avidu 197609 486 Jun 10 16:29 MahadevpuraSingles.summary.md
-rw-r--r-- 1 avidu 197609 1.5G May 4 20:39 TestLargeVideo.MP4
-rw-r--r-- 1 avidu 197609 287 Jun 8 21:33 TestLargeVideo.rallies.csv
-rw-r--r-- 1 avidu 197609 6.6G May 22 08:40 gBaaddy20May2026.MP4
-rw-r--r-- 1 avidu 197609 2.1K Jun 8 21:53 gBaaddy20May2026.rallies.csv
```

30. user (2026-06-13T19:55:58.848Z)

107 C:/Users/avidu/Projects/sports-data-collector/data/eval_export/manifest.json

31. user (2026-06-13T19:55:59.711Z)

```
backend\pipeline\segmenters\__init__.py:7:from backend.pipeline.segmenters.fusion_hybrid import
FusionSegmenter
backend\pipeline\segmenters\__init__.py:17: "FusionSegmenter",
backend\pipeline\segmenters\fusion_hybrid.py:23:nothing is gated (thresholds 0, person OFF), so
`FusionSegmenter` with default config seeds
backend\pipeline\segmenters\fusion_hybrid.py:39:class
FusionSegmenter(TrajectoryHybridSegmenter):
backend\pipeline\segmenters\fusion_hybrid.py:89:
stats.update(self._person_features(cw, points, fps, frame_width, video_path, fcfg))
backend\pipeline\segmenters\fusion_hybrid.py:103: def _person_features(self, cw: Dict[str,
Any], points: List[Any], fps: float,
backend\pipeline\segmenters\fusion_hybrid.py:118: det =
self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
backend\pipeline\segmenters\fusion_hybrid.py:130: return
ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
backend\pipeline\segmenters\fusion_hybrid.py:154:segmenter_registry.register(FusionSegmenter)
backend\pipeline\segmenters\trajectory_hybrid.py:87: rows; ``FusionSegmenter`` overrides
this to add ``net_crossings`` (+ person-cue
backend\pipeline\segmenters\trajectory_hybrid.py:101: ``FusionSegmenter`` overrides this
to apply Gate A/B when thresholds are raised.
```

32. assistant (2026-06-13T19:56:01.114Z)

Now let me read the FusionSegmenter code to understand the full flow:

33. user (2026-06-13T19:56:01.529Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
```

```

10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13     torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54             @property
55             def name(self) -> str:
56                 return "fusion_hybrid"
57
58             @property
59             def description(self) -> str:
60                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate

```

```

rallies, "
61         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62         "AI provider sets semantic boundaries.")
63
64     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66         if substrate == "tracknet":
67             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68             return TrackNetRunner(cfg), {
69                 "weights_path": cfg.weights_path,
70                 "queue_length": cfg.queue_length,
71                 "fusion_substrate": "tracknet",
72             }
73         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                frame_width: float, video_path: str,
78                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                          frame_width: float, video_path: str,
105                          fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.

```

```

110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

34. user (2026-06-13T19:56:01.765Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family``      ? config section under ``indexing.*``, output subdir, and

```

```

12         the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16         detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:

```

```

75         return max(3, int(duration / 0.8))
76     try:
77         return int(shot_count)
78     except (ValueError, TypeError):
79         return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                      player_pool: Optional[List[str]] = None,
113                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---
119         sport_name = self.config.get("sport", "badminton")
120         try:
121             sport_profile = sport_registry.get(sport_name)
122         except KeyError as e:
123             logger.error(str(e))
124             return []
125
126         idx_cfg = self.config.get("indexing", {})
127         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
128 "gemini"))
129         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
130 "gemini-2.5-flash"))
131
132         api_key_env_var = f"{provider_name.upper()}_API_KEY"
133         api_key = os.environ.get(api_key_env_var)
134         if not api_key:
135             logger.error(
136                 f"{api_key_env_var} environment variable is not set! "
137                 f"To run the {provider_name} handoff, set your API key. "
138                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
139         )

```

```

133         return []
134     try:
135         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136     except KeyError as e:
137         logger.error(str(e))
138         return []
139
140     video_duration = get_video_duration(video_path)
141     if video_duration <= 0:
142         logger.error("Invalid video duration.")
143         return []
144     fps, frame_width = self._probe_fps_width(video_path)
145
146     # --- Runner config + windowing thresholds ---
147     runner, runner_params = self._build_runner(idx_cfg)
148
149     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

```

35. user (2026-06-13T19:56:06.783Z)

GPU available: True
GPU: NVIDIA GeForce RTX 2070 Super with Max-Q Design
VRAM: 8.0GB

36. assistant (2026-06-13T19:56:08.411Z)

Perfect. Now let me get performance benchmarks and check for Modern Standby handling in the code.

37. user (2026-06-13T19:56:25.583Z)

Downloading: "https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_coco-258fb6c6.pth" to C:\Users\avidu\.cache\torch\hub\checkpoints\fasterrcnn_resnet50_fpn_coco-258fb6c6.pth

```

0%|          | 0.00/160M [00:00<?, ?B/s]
2%|?         | 3.38M/160M [00:00<00:04, 35.2MB/s]
5%|?         | 7.62M/160M [00:00<00:04, 37.8MB/s]
9%|?         | 14.1M/160M [00:00<00:02, 51.1MB/s]
12%|??       | 19.1M/160M [00:00<00:02, 50.8MB/s]
15%|??       | 24.1M/160M [00:00<00:03, 44.9MB/s]
18%|??       | 28.6M/160M [00:00<00:03, 42.1MB/s]
21%|??       | 32.8M/160M [00:00<00:03, 37.9MB/s]
23%|???      | 36.5M/160M [00:00<00:03, 33.1MB/s]
25%|???      | 39.9M/160M [00:01<00:04, 30.2MB/s]
27%|???      | 42.9M/160M [00:01<00:04, 29.2MB/s]
29%|???      | 45.8M/160M [00:01<00:04, 27.9MB/s]
30%|???      | 48.5M/160M [00:01<00:04, 26.7MB/s]
32%|????     | 51.1M/160M [00:01<00:04, 25.6MB/s]
34%|????     | 53.8M/160M [00:01<00:04, 25.9MB/s]
35%|????     | 56.2M/160M [00:01<00:04, 24.8MB/s]
37%|????     | 59.1M/160M [00:01<00:04, 24.8MB/s]
39%|????     | 61.6M/160M [00:02<00:04, 25.0MB/s]
40%|????     | 64.4M/160M [00:02<00:03, 26.0MB/s]
42%|??????   | 66.9M/160M [00:02<00:03, 24.4MB/s]
44%|??????   | 69.6M/160M [00:02<00:03, 25.5MB/s]
45%|??????   | 72.1M/160M [00:02<00:03, 24.8MB/s]
47%|??????   | 74.6M/160M [00:02<00:03, 24.9MB/s]
48%|??????   | 77.2M/160M [00:02<00:03, 24.7MB/s]
50%|??????   | 79.6M/160M [00:02<00:03, 24.6MB/s]
51%|???????  | 82.1M/160M [00:02<00:03, 24.5MB/s]
53%|???????  | 84.6M/160M [00:03<00:03, 24.4MB/s]
55%|???????  | 87.2M/160M [00:03<00:03, 24.5MB/s]

```

56%|?????? | 89.8M/160M [00:03<00:02, 25.0MB/s]
58%|?????? | 92.2M/160M [00:03<00:02, 25.1MB/s]
59%|?????? | 94.8M/160M [00:03<00:02, 25.3MB/s]
61%|?????? | 97.2M/160M [00:03<00:02, 25.1MB/s]
63%|?????? | 99.9M/160M [00:03<00:02, 24.7MB/s]
64%|?????? | 102M/160M [00:03<00:02, 24.8MB/s]
66%|?????? | 105M/160M [00:03<00:02, 24.7MB/s]
67%|?????? | 107M/160M [00:03<00:02, 24.5MB/s]
69%|?????? | 110M/160M [00:04<00:02, 25.0MB/s]
70%|?????? | 112M/160M [00:04<00:02, 23.9MB/s]
72%|?????? | 115M/160M [00:04<00:01, 24.9MB/s]
74%|?????? | 118M/160M [00:04<00:01, 25.3MB/s]
75%|?????? | 120M/160M [00:04<00:01, 25.1MB/s]
77%|?????? | 123M/160M [00:04<00:01, 24.9MB/s]
78%|?????? | 125M/160M [00:04<00:01, 24.7MB/s]
80%|?????? | 128M/160M [00:04<00:01, 24.4MB/s]
81%|?????? | 130M/160M [00:04<00:01, 24.9MB/s]
83%|?????? | 133M/160M [00:05<00:01, 24.9MB/s]
85%|?????? | 135M/160M [00:05<00:01, 25.0MB/s]
86%|?????? | 138M/160M [00:05<00:00, 25.3MB/s]
88%|?????? | 140M/160M [00:05<00:00, 25.3MB/s]
89%|?????? | 143M/160M [00:05<00:00, 23.8MB/s]
91%|?????? | 146M/160M [00:05<00:00, 25.1MB/s]
93%|?????? | 148M/160M [00:05<00:00, 25.0MB/s]
94%|?????? | 150M/160M [00:05<00:00, 24.4MB/s]
96%|?????? | 153M/160M [00:05<00:00, 25.1MB/s]
97%|?????? | 156M/160M [00:06<00:00, 24.2MB/s]
99%|?????? | 158M/160M [00:06<00:00, 24.7MB/s]
100%|?????? | 160M/160M [00:06<00:00, 27.0MB/s]

Model: fasterrcnn_resnet50_fpn
Total params: 41.8M
Device: cuda
Weights loaded from COCO (BSD license)

38. assistant (2026-06-13T19:56:27.419Z)

Excellent. Now let me check the golden video CSVs and estimate frame counts:

39. user (2026-06-13T19:56:31.540Z)

Adarsh and Avi.MP4: 10.7GB @ 59.94fps, 95 rallies
KushagraYashAviFirstVideo.MP4: 7.5GB @ 59.94fps, 31 rallies
LargeTestVideo.MP4: 10.7GB @ 29.97fps, 48 rallies
gBaaddy20May2026.MP4: 6.6GB @ 59.94fps, 47 rallies
TestLargeVideo.MP4: 1.4GB @ 29.97fps, 5 rallies
MahadevpuraSingles.MP4: 0.2GB @ 29.97fps, 30 rallies

Total: 37.1GB, 256 rallies across 6 videos

40. assistant (2026-06-13T19:56:33.063Z)

Now let me estimate frame counts and GPU memory/time costs:

41. user (2026-06-13T19:56:36.676Z)

Video frame counts and durations:

Adarsh and Avi.MP4		84,480 frames		23.5min		1920x1080		sampled: 28,160
KushagraYashAviFirstVideo.MP4		32,232 frames		9.0min		3840x2160		sampled: 10,744
LargeTestVideo.MP4		23,040 frames		12.8min		5312x2988		sampled: 7,680
gBaaddy20May2026.MP4		52,232 frames		14.5min		1920x1080		sampled:

```
17,411
TestLargeVideo.MP4          |    3,019 frames |    1.7min | 5312x2988 | sampled:
1,007
MahadevpuraSingles.MP4     |   13,418 frames |    7.5min | 1920x1080 | sampled:
4,473
-----
Total frames: 208,421
Approx duration at 30fps: 115.8 minutes
With frame_skip=3: ~69,473 sampled frames
```

42. assistant (2026-06-13T19:56:38.589Z)

Now let me check for any Modern Standby keep-alive patterns and look at the actual cost model:

43. user (2026-06-13T19:58:40.243Z)

Command running in background with ID: b7bst5flh. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\tasks\b7bst5flh.output. You will be notified when it completes. To check interim output, use Read on that file path.

44. user (2026-06-13T19:58:44.198Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md

45. user (2026-06-13T19:58:45.748Z)

4. ****Throttle bulk runs**** (`--sleep` + per-video `--retries`/backoff) and treat a

46. assistant (2026-06-13T19:58:47.161Z)

Now let me create a comprehensive GPU recipe document by examining benchmark expectations and the actual test patterns:

47. user (2026-06-13T19:59:00.896Z)

```
=====
P3 PERSON DETECTION COST ESTIMATE (RTX 2070 Super, 8GB, frame_skip=3)
=====
```

Model: fasterrcnn_resnet50_fpn

- Parameters: 41.8M
- Weights: 160MB (COCO, BSD license)
- Device: CUDA (RTX 2070 Super, 8GB VRAM)

Frame processing:

- Total raw frames (6 golden videos): 208,421
- Sampled frames (frame_skip=3): 69,473
- Est. inference time: 50ms/frame (50ms avg, includes ByteTrack)

Estimated total time:

- 57.9 minutes (0.96 hours)
- At 100% GPU utilization with no overhead

Practical wall-clock time (with I/O, overhead, tracker state):

- 1.5x margin: 86.8 minutes (1.45 hours)
- 2x margin (safe): 115.8 minutes (1.93 hours)

Memory footprint:

- Model weights (VRAM): 160MB
- Activations + batch (1 frame at a time): ~300-400MB
- ByteTrack history/frame: ~50MB
- Total resident: ~600MB (well within 8GB)
- GPU margin: 7.4GB free (safe)

Persisted output (to DB `candidate\_segments.stats`):

- Candidate windows: ~250
- 5 person features each: 5
- Est. JSON size: 0.0MB

=====

48. assistant (2026-06-13T19:59:02.267Z)

Now I have all the information. Let me create the comprehensive GPU recipe report:

49. user (2026-06-13T19:59:49.829Z)

Structured output provided successfully

50. assistant (2026-06-13T20:00:00.958Z)

Perfect! I've completed a comprehensive GPU-run recipe for P3 person detection on your RTX 2070 Super. Here's the summary:

Key Deliverables

****Realistic Cost/Time:****

- ****58 minutes pure inference**** (69,473 sampled frames at 50ms/frame with `frame\_skip=3`)
- ****90?120 minutes practical wall-clock**** (including I/O, tracker overhead, Python overhead)
- ****Memory safe:**** 600MB resident on 8GB VRAM ? 7.4GB margin; zero OOM risk

****Golden Trajectory Reuse (NO WASB RE-RUN NEEDED):****

- Existing WASB CSVs in `output/wasb/\*\_wasb.csv` are your golden trajectory substrate
- Person detection runs ****over**** these fixed windows via `PersonDetector.detections\_per\_frame()` (person_detector.py:258?303)
- Features persisted to `candidate\_segments.stats` (5 keys: `players\_in\_court`, `shuttle\_player\_colocation`, etc.)

****Recommended Path (Lightweight Offline):****

- ****NOT**** the full FusionSegmenter+Gemini pipeline
- Use `PersonDetector` to extract person features over existing candidates (GPU 1?2 hrs)
- Then experiment with thresholds (`min\_players`, `min\_crossings`) offline via `experiment.py`
- This avoids re-running WASB and avoids Gemini API costs for feature tuning

****Modern Standby Mitigation:****

- Windows 11 Pro's S0i3 can interrupt long runs despite High Performance mode
- Solution: `keep\_awake.ps1` (NumLock ping every 2 min) + run person detection in WSL2 detached from Windows
- Or disable Modern Standby system-wide via `powercfg` (documented in report)

****Monitoring & Hygiene:****

- Watch `nvidia-smi` + tail logs during run
- Poll DB candidate count every 30 sec to track progress
- Archive results to timestamped backup before next experiment
- Keep old `\*\_wasb.csv` files (they're golden trajectories, reused)

****File Locations (Quoted):****

- Person detector: `backend/pipeline/detectors/person\_detector.py:50?303`
- FusionSegmenter orchestration: `backend/pipeline/segmenters/fusion\_hybrid.py:39?154`
- Feature math (pure, GPU-free): `backend/pipeline/detectors/fusion\_features.py:192?216`
- Golden videos: `C:\Users\avidu\Projects\Annotation Setup\Collect\Golden Labelled` (6 videos, 37.1GB, 208,421 frames)
- Experiment harness: `backend/eval/experiment.py:73?143` (Variant, compare, no regression gate)

The report in the structured output above includes exact entrypoints, sample invocations, gotchas (ByteTrack chunk-local IDs, multi-court confound), and a complete reference grid keyed to file:line.